ECE 374 B: Algorithms and Models of Computation, Summer 2026

Midterm 3 – August 04, 2025

- **You will have 75 minutes (1.25 hours) to solve all the problems. Most have multiple parts.** Don't spend too much time on questions you don't understand and focus on answering as much as you can!
- ***BUDGET YOUR TIME WISELY***. I highly recommend working on the questions you know first and the questions you need to think about second.
- *No* resources are allowed for use during the exam except a multi-page cheatsheet and scratch paper on the back of the exam. ***Do not tear out the cheatsheet or the scratch paper!*** It messes with the auto-scanner.
- You should write your answers *completely* in the space given for the question. We will not grade parts of any answer written outside of the designated space.
- Please *use a dark-colored pen* unless you are *absolutely* sure your pencil writing is forceful enough to be legible when scanned. We reserve the right to take off points if we have difficulty reading the uploaded document.
- Unless otherwise stated, assume $\mathsf{P} \neq \mathsf{NP}$.
- Assume that whenever the word "reduction" is used, we mean a (not necessarily polynomial-time) *mapping/many-one* reduction.
- You can only refer to the cheat sheet content as a black box.
- ***Don't cheat.*** If we catch you, you will get an F in the course.
- ***Good luck!***

Name: _____

NetID: _____

Date: _____

1 Short Answer I - 28 points

For each of the problems circle *true* if the statement is *always* true, circle *false* otherwise. There is no partial credit for these questions.

(a) NP stands for "non-polynomial" time.

True False

(b) Every problem in P is also in NP.

True False

(c) If L is an NP-complete language and $L \in P$, then $P = NP$

True False

(d) There exists a polynomial time reduction from every NP-hard problem to every problem in P.

True False

(e) Every NP-hard problem must be decidable.

True False

(f) If a problem is NP-complete, then it must be a decision problem.

True False

(g) If there is a polynomial time reduction from problem A to problem B and B is in NP, then A must also be in NP.

True False

(h) If a problem can be solved in polynomial space, then it is also solvable in polynomial time.

True False

(i) If a problem is solvable in polynomial time, then it must also be solvable in polynomial space.

True False

(j) There are problems (X) that are in NP for which the reduction $X \leq 3SAT$ cannot be shown.

True False

(k) A problem that requires exponential time must also require exponential space.

True False

(l) All NP-hard problems must have yes/no outputs.

True False

(m) If L is decidable, then $\overline{L}$ must be also be decidable:

True False

(n) If L is turing-recognizable but not decidable, then $\overline{L}$ must be Turing-unrecognizable:

True False

2 Short Answer III - 12 points

Assuming the reductions below can be proven, circle all the classes that the problem X *may* belong to (no explanation, this is just multiple choice) :

- $CLIQUE \leq_P X$

P NP NP-hard NP-complete

decidable undecidable

- $X \leq_P Longest\_increasing\_subsequence$

P NP NP-hard NP-complete

decidable undecidable

- $A_{TM} \implies X$

P NP NP-hard NP-complete

decidable undecidable

3 Classification I (P/NP) - 12 points

Is the following problem in P, NP, or some combinations of complexity classes? For each of the following problems, circle all the complexity classes that problem belongs to. Whatever class it is in, prove it!

FewerThanKPath (FTKP) problem - You are given a directed, acyclic graph with weighted edges and need to find if there is a path from s to t with a weight less than k.

- Input: A weighted (positive & negative) directed acyclic graph G, start/end nodes s/t, and a number k.
- Output: True if there exists a path from s to t with a weight less than k. False otherwise.

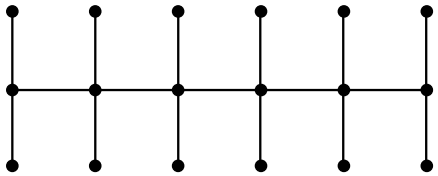
Which of the following complexity classes does this problem belong to? Circle ***all*** that apply:

P NP NP-hard NP-complete

4 Classification II (P/NP) - 12 points

Is the following problem in P, NP, or some combinations of complexity classes? For each of the following problems, circle all the complexity classes that problem belongs to. Whatever class it is in, prove it!

A centipede is an undirected graph formed by a path of length k with two edges (legs) attached to each node on the path as shown in the below figure. Hence, the centipede graph has 3k vertices.



CENTIPEDE graph with $k = 6$

The CENTIPEDE problem asks whether there exists a centipede subgraph of $3k$ vertices :

- Input: A undirected graph $G = V, E$ and integer k
- Output: True if there does exist a CENTIPEDE graph. False otherwise.

Which of the following complexity classes does this problem belong to? Circle ***all*** that apply:

P NP NP-hard NP-complete

5 Classification I (Decidability) - 12 points

Are the following languages decidable? For each of the following languages,

- Circle one of "decidable" or "undecidable" to indicate your choice.
- If you choose "decidable", show your choice correct by describing an algorithm that decides that language. If you choose "undecidable", show your choice correct by giving a reduction proving its correctness.
- Regardless of your choice, explain *briefly* (i.e., in few sentences, diagrams, *clear* pseudo-code) why your choice is valid.

AcceptNoHomers$_{TM} = \{\langle M \rangle \mid M$ is a ***TM*** and does not accept any strings that contain "Homer" as a substring$\}$

decidable undecidable

6 Classification II (Decidability) - 12 points

Are the following languages decidable? For each of the following languages,

- Circle one of "decidable" or "undecidable" to indicate your choice.
- If you choose "decidable", show your choice correct by describing an algorithm that decides that language. If you choose "undecidable", show your choice correct by giving a reduction proving its correctness.
- Regardless of your choice, explain *briefly* (i.e., in few sentences, diagrams, *clear* pseudo-code) why your choice is valid.

$$\text{AcceptSlowly}_{TM} = \{\langle M, w\rangle \mid M \text{ is a } \boldsymbol{TM} \text{ and accepts } w \text{ in } 2^{|w|} \text{ steps.}\}$$

decidable undecidable

7 Classification (Mixed) - 12 points

Same type of problem as before, except now you have to choose between a mix of computational and algorithmic complexity classes.

In the **MinVertexCover problem**, you are given an undirected graph and asked to find the size of the smallest set of vertices that touches every edge.

- Input: An unweighted undirected graph $G = (V, E)$.
- Output: A value x denoting the minimum number of vertices needed to cover every edge in G.

Which of the following classes does this problem belong to? Whatever you choose, *succinctly* prove it!

P NP NP-hard NP-complete decidable undecidable

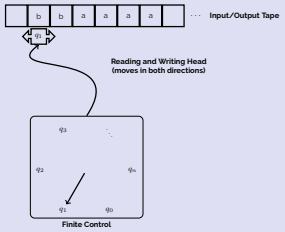
This page is for additional scratch work!

ECE 374 B Reductions, P/NP, and Decidability: Cheatsheet

Turing Machines

Turing machine is the simplest model of computation.

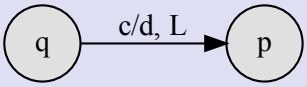
- Input written on (infinite) one sided tape.
- Special blank characters.
- Finite state control (similar to DFA).
- Ever step: Read character under head, write character out, move the head right or left (or stay).
- Every TM **M** can be encoded as a string $\langle M \rangle$



Transition Function: $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\leftarrow, \rightarrow, \square\}$

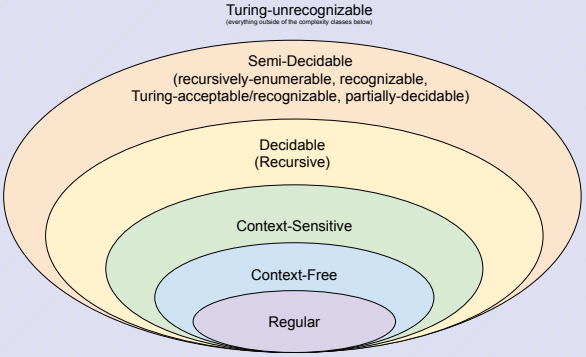
$\delta(q, c) = (p, d, \leftarrow)$

- q: current state.
- c: character under tape head.
- p: new state.
- d: character to write under tape head
- $\leftarrow$: Move tape head left.

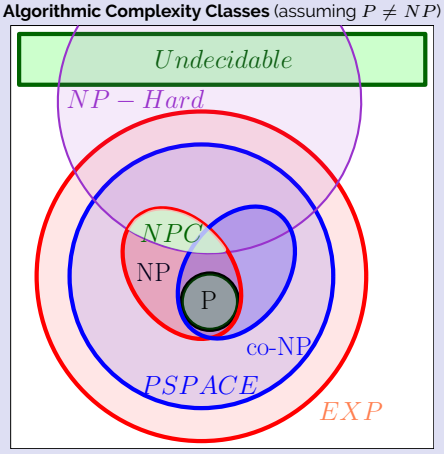


Complexity Classes

Computational Complexity Classes



Algorithmic Complexity Classes (assuming $P \neq NP$)



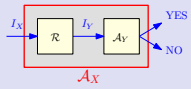
Reductions

A general methodology to prove impossibility results.

- Start with some *known* hard problem X
- *Reduce* X to your favorite problem Y

If Y can be solved then so can $X \implies Y$. But we know X is hard so Y has to be hard too. On the other hand if we know Y is easy, then X has to be easy too.

The Karp reduction, $X \leq_P Y$ suggests that there is a polynomial time reduction from X to Y.



Assuming

- $R(n)$: running time of $\mathcal{R}$
- $Q(n)$: running time of $\mathcal{A}_Y$

Running time of $\mathcal{A}_X$ is $O(Q(R(n)))$

Sample NP-complete problems

CircuitSat:	Given a boolean circuit, are there any input values that make the circuit output True?
3Sat:	Given a boolean formula in conjunctive normal form, with exactly three distinct literals per clause, does the formula have a satisfying assignment?
IndependentSet:	Given an undirected graph G and integer k, what is there a subset of vertices $\geq k$ in G that have no edges among them?
Clique:	Given an undirected graph G and integer k, is there a complete complete subgraph of G with more than k vertices?
kPartition:	Given a set X of kn positive integers and an integer k, can X be partitioned into n, k-element subsets, all with the same sum?
3Color:	Given an undirected graph G, can its vertices be colored with three colors, so that every edge touches vertices with two different colors?
HamiltonianPath:	Given graph G (either directed or undirected), is there a path in G that visits every vertex exactly once?
HamiltonianCycle:	Given a graph G (either directed or undirected), is there a cycle in G that visits every vertex exactly once?
LongestPath:	Given a graph G (either directed or undirected, possibly with weighted edges) and an integer k, does G have a path $\geq k$ length?

- Remember a **path** is a sequence of distinct vertices $[v_1, v_2, \ldots v_k]$ such that an edge exists between any two vertices in the sequence. A **cycle** is the same with the addition of a edge $(v_k, v_1) \in E$. A **walk** is a path except the vertices can be repeated.
- A formula is in conjunction normal form if variables are or'ed together inside a clause and then clauses are and'ed together: $((x_1 \vee x_2 \vee x_3) \wedge (\overline{x_2} \vee x_4 \vee x_5))$. Disjunctive normal form is the opposite $((x_1 \wedge x_2 \wedge x_3) \vee (\overline{x_2} \wedge x_4 \wedge x_5))$.

Sample undecidable problems

AcceptOnInput:	$A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ accepts on } w\}$
HaltsOnInput:	$Halt_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM and halts on input } w\}$
HaltOnBlank:	$HaltB_{TM} = \{\langle M\rangle \mid M \text{ is a TM \& } M \text{ halts on blank input}\}$
Emptiness:	$E_{TM} = \{\langle M\rangle \mid M \text{ is a TM and } L(M) = \varnothing\}$
Equality:	$EQ_{TM} = \left\{\langle M_A, M_B\rangle \;\middle|\; \begin{array}{l} M_A \text{ and } M_B \text{ are TM's} \\ \text{and } L(M_A) = L(M_B) \end{array}\right\}$